



Configuring Space Profiles for K2cloud Non-Development Customer Deployments

Technical Note

Date: 11 April 2025

Version: 1.0

This document contains copyrighted work and proprietary information belonging to K2view. All intellectual property rights (including, without limitation, copyrights, trade secrets, trademarks, etc.) evidenced by or embodied in and/or attached, connected, or related to this document, as well as any information contained herein, are and shall be owned solely by K2view. K2view does not convey to you an interest in or to this document, the information contained herein, or its intellectual property rights, but only a personal, limited, fully revocable right to use the document solely for review purposes. Unless explicitly set forth otherwise, you may not reproduce by any means any document and/or copyright contained herein.

Information in this document is subject to change without notice. Corporate and individual names and data used in the examples herein are fictitious unless otherwise noted.

Copyright © 2025 K2view Ltd. / K2VIEW LLC. All rights reserved.

The following are trademarks of K2view: K2view logo, K2view's platform.

K2view reserves the right to update this list document from time to time.

Table of Contents

1.	Background	2
2.	Information Gathering	2
2.1.	Replication Factor	2
2.2.	Disk Size and Storage Configuration	2
2.3.	Resource Management & Runtime Settings	3
2.4.	Autoscaling (scaling)	3
2.5.	Secrets Configuration	3

1. Background

When a new K2view Space is provisioned using a K2cloud Space Profile, it serves as a blueprint with default configurations for memory, CPU, storage, replication, and service enablement, established by the K2cloud Agent. Although our defaults function well for most environments, production, staging, and development workloads often have unique operational and resource needs.

This is why we proactively ask customers about their preferences for the space profile we tailor, especially in production environments. This ensures proper sizing and prevents overcommitment.

Here is some of this information.

2. Information Gathering

2.1. Replication Factor

Development environments typically require only one replica. In production, multiple replicas may be necessary for high availability.

- We ask: “Do you want more than one Fabric node deployed in the space? If so, how many?”

Specifies the number of Fabric pod instances deployed in the space.

- 1: A single instance is deployed.
- Typically used in **Dev** and **QA** environments to reduce complexity and cost.

In production, multiple replicas may be preferred to avoid a single point of failure.

2.2. Disk Size and Storage Configuration

Storage in cloud environments is often immutable once attached, especially for PVC-backed volumes.

- We ask: “What is your expected disk usage for caching or temporary files?”

If you don’t have a preference, we will set the default to 500MB. Remember that developers may misuse features that use large amounts of disk space.

2.3. Resource Management & Runtime Settings

Defines the CPU and memory allocation for each Fabric pod operating in the space. This ensures that each Fabric instance gets predictable compute while avoiding noisy-neighbor issues in shared environments.

Parameter	Description
requests.cpu	The minimum guaranteed CPU allocation. The pod is scheduled on a node that can provide at least this much CPU. Default: 1 vCPU .
requests.memory	The minimum guaranteed memory allocation. The pod receives at least a default of 8 GiB RAM .
limits.cpu	The maximum CPU the pod is allowed to consume. Enforced by Kubernetes. Default: 2 vCPU .
limits.memory	The maximum RAM usage allowed before the pod is throttled or terminated. Default: 16 GiB RAM .

2.4. Autoscaling (scaling)

Defines horizontal pod autoscaling (HPA) settings. This is disabled by default and recommended.

Parameter	Description
enabled	Whether HPA is active (false = disabled).
minReplicas	Minimum number of pods when scaling (1).
maxReplicas	Maximum number of pods during scale-out (2).
targetCPU	CPU utilization threshold for scaling out (in %).

Since autoscaling is **disabled**, only the initial replica count applies. This can be enabled later as the workload demands increase.

2.5. Secrets Configuration

Injects environment-specific secret values into the container at runtime.

Key	Description
MAX_HEAP	Maximum Java heap size (used by Fabric runtime). Default: 4G.

These values are passed as environment variables or mounted files, allowing for flexible customization per space.